

Sistema Inteligente de Rotas para a Rede Metroviária de São Paulo

Modelagem de redes metroviárias utilizando algoritmos de grafos

Trabalho da Disciplina de Estrutura de Dados

Estudantes: Deborah Silva Sande, Eduardo Menezes de Oliveira, Enzo Gabriel de Jesus Salermo, Felipe Sande Barreto, Gustavo Celestino Lima de Jesus e Lincoln Meira de Santana

Orientadores: Prof. Dr. Tiago Palma Pagano

Cursos: Bacharelado em Ciências Exatas e Tecnológicas e Engenharia de Computação

Sumário

- 1 Resumo
- 2 Contexto
- 3 Justificativa
- 4 Objetivos
- 5 Metodologia
 - Modelagem em Grafos
 - Metodologia — Algoritmos Utilizados
 - Metodologia — Estrutura do Sistema
 - Metodologia — Modelagem dos Dados
 - Metodologia — Implementação dos Algoritmos
 - Metodologia — Critérios Alternativos
 - Metodologia — Interface Gráfica
 - Metodologia — Funcionalidades do sistema
- 6 Resultados
- 7 Conclusão
 - Considerações Finais
 - Limitações
 - Trabalhos Futuros

Resumo

O transporte metroferroviário é um dos principais meios de mobilidade urbana em grandes cidades, tornando importante o planejamento eficiente de rotas. Nesse contexto, este trabalho apresenta o desenvolvimento de um sistema de rotas inteligentes para a rede metroferroviária de São Paulo, implementado em Java, que modela a rede como um grafo ponderado. As estações são representadas como vértices e as conexões como arestas ponderadas pelo tempo de deslocamento, permitindo a implementação dos algoritmos de Dijkstra, Busca em Largura (BFS), Busca em Profundidade (DFS) e Floyd-Warshall, desenvolvidos manualmente, sem o uso de bibliotecas externas de grafos. O sistema permite calcular o menor caminho, simular interdições de estações, identificar as estações mais importantes da rede e disponibiliza uma interface gráfica desenvolvida em Swing. Dessa forma, foi constatado que a Teoria dos Grafos possibilita a resolução de problemas de mobilidade urbana.

Palavras-chave: Mobilidade Urbana; Rotas Inteligentes; Grafos; Dijkstra; Floyd-Warshall.

Contexto

- O transporte metroferroviário é um dos principais meios de mobilidade urbana em grandes cidades [1].
- Encontrar a melhor rota pode envolver diferentes critérios, como tempo de viagem, número de baldeações e interdições de estações.
- A Teoria dos Grafos permite modelar estações como vértices e conexões como arestas [2, 3].
- Neste trabalho foi desenvolvido um sistema em *Java* que utiliza algoritmos clássicos de grafos para calcular rotas, simular interdições e analisar a importância das estações da rede metroferroviária de São Paulo.

Justificativa

- Redes metroferroviárias podem ser modeladas como grafos, em que as estações são representadas por vértices e as conexões por arestas.
- Essa representação permite aplicar algoritmos de caminho mínimo e de busca para resolver problemas de roteamento e análise da rede [4].
- Algoritmos como Floyd-Warshall possibilitam calcular os menores caminhos entre todos os pares de vértices da rede [5, 6].
- Assim, este trabalho aplica esses algoritmos em um sistema desenvolvido em Java para demonstrar sua utilização em um problema prático de mobilidade urbana [3].

Objetivos

Objetivo Geral

Desenvolver um sistema de rotas inteligentes para redes metroferroviárias, utilizando a Teoria dos Grafos e algoritmos clássicos para consulta e análise de trajetos.

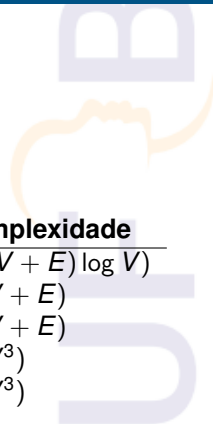
Objetivos Específicos

- Modelar a rede metroferroviária como um grafo ponderado.
- Implementar os algoritmos Dijkstra, BFS, DFS e Floyd-Warshall.
- Determinar rotas com menor tempo, menor número de estações e menor número de baldeações.
- Simular a interdição de estações e recalcular os trajetos.
- Desenvolver uma interface gráfica para visualização da rede e das rotas.

E) $[2, 3]$, em
ou pontos
timado em
estação.

7 / 27

Algoritmos Utilizados



Algoritmo	Função	Complexidade
Dijkstra	Menor tempo	$O((V + E) \log V)$
BFS	Menor número de estações	$O(V + E)$
DFS	Conectividade e ciclos	$O(V + E)$
Floyd-Warshall	Caminhos entre todos os pares	$O(V^3)$
Betweenness	Importância estrutural	$O(V^3)$

Os algoritmos foram implementados diretamente no projeto para fins didáticos e de controle da lógica interna [5, 6].

Algoritmo de Dijkstra

Animação ilustrativa do algoritmo de Dijkstra para encontrar o caminho de menor custo.

Fonte: Elgabry, O. *Path Finding Algorithms*. Disponível em:

<https://medium.com/omarelgabrys-blog/path-finding-algorithms-f65a8902eb40>. Acesso em: 16 jul. 2026.

Algoritmo BFS

Animação ilustrativa do algoritmo BFS para percorrer o grafo em níveis e encontrar caminhos com o menor número de arestas.

Fonte: Elgabry, O. *Path Finding Algorithms*. Disponível em:

<https://medium.com/omarelgabrys-blog/path-finding-algorithms-f65a8902eb40>. Acesso em: 16 jul. 2026..

Algoritmo DFS

Animação ilustrativa do algoritmo DFS para explorar em profundidade cada ramo do grafo antes de retroceder.

Fonte: Elgabry, O. *Path Finding Algorithms*. Disponível em:

<https://medium.com/omarelgabrys-blog/path-finding-algorithms-f65a8902eb40>. Acesso em: 16 jul. 2026..

Algoritmo de Floyd-Warshall

Animação ilustrativa do algoritmo de Floyd-Warshall para encontrar os menores caminhos entre todos os pares de vértices do grafo.

Fonte: Wikimedia Commons. *Floyd warshall gif.gif*. Disponível em:

https://commons.wikimedia.org/wiki/File:Floyd_warshall_gif.gif. Acesso em: 16 jul.

2026.

Algoritmo Betweenness Centrality

Animação ilustrativa da centralidade de intermediação para identificar os vértices que aparecem com maior frequência nos menores caminhos do grafo.

Fonte: Medium. *Betweenness Centrality Graph Algorithm: What, Why and How*. Disponível em:
[https://medium.com/@flujoyittjaiemqupd/
betweenness-centrality-graph-algorithm-what-why-and-how-d555e4458c0d](https://medium.com/@flujoyittjaiemqupd/betweenness-centrality-graph-algorithm-what-why-and-how-d555e4458c0d). Acesso em:
17 jul. 2026.



Modelagem dos Dados

A base de dados do projeto representa a rede metroferroviária de São Paulo com 14 linhas e 214 vértices estação-linha[7, 8].

- Cada estação possui id, nome, linha e status de atividade.
- Estações de baldeação são representadas como vértices distintos por linha.
- Conexões entre estações recebem pesos entre 2 e 4 minutos.
- Integrações recebem pesos estimados de 3 ou 5 minutos.

Essa modelagem preserva a informação da linha durante o cálculo da rota.

Rota por Menor Tempo

O algoritmo de Dijkstra calcula o caminho de menor custo entre duas estações, considerando o peso das conexões como tempo estimado de deslocamento [4, 9].

- Inicializa a distância da origem como zero.
- Usa fila de prioridade para escolher o vértice de menor custo acumulado.
- Relaxa as arestas dos vizinhos ativos.
- Reconstrói a rota usando o mapa de predecessores.
- Conta as baldeações a partir das mudanças de linha no caminho final.

Rotas por Critérios Alternativos

Além da rota por menor tempo, o sistema implementa estratégias complementares para atender diferentes necessidades do usuário.

Menor número de estações

Utiliza BFS para minimizar a quantidade de arestas percorridas entre origem e destino[3].

Menor número de baldeações

Utiliza comparação lexicográfica: primeiro minimiza trocas de linha e depois usa o tempo como critério de desempate.

Interface Gráfica

A aplicação `MapaMetroSPApp.java` fornece uma interface em *Swing* para demonstração interativa da rede.

- Clique esquerdo seleciona origem e destino.
- Clique direito interdita ou reativa uma estação.
- A rota calculada é destacada visualmente no mapa.
- Tooltips exibem informações sobre estações.
- Diálogo específico apresenta as estações mais importantes.

O posicionamento é gerado por um layout de forças simplificado, baseado na topologia da rede.

Funcionalidades do Sistema

O sistema alcançou os principais requisitos funcionais definidos para a pesquisa:

- Cadastro programático da rede com 14 linhas e 214 vértices.
- Cálculo de rotas por menor tempo usando Dijkstra.
- Busca por menor número de estações usando BFS.
- Cálculo de rotas com menor número de baldeações.
- Simulação de falhas operacionais por interdição de estações.
- Visualização interativa com destaque do percurso calculado.

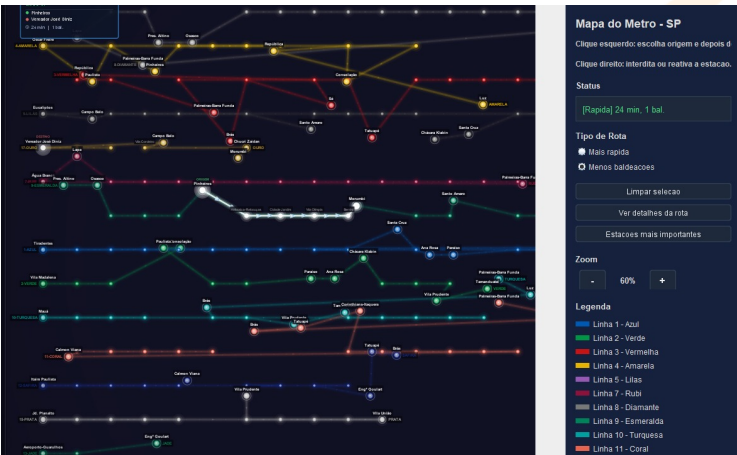
Resultados

Figura 1 – Interface gráfica do sistema desenvolvido



Resultados

Figura 2 – Interface gráfica do sistema desenvolvido



Fonte: Autoria própria.

Considerações Finais

O projeto demonstrou a viabilidade de representar uma rede metroviária como grafo e aplicar algoritmos clássicos para resolver consultas de rota.

A implementação em Java evidencia a integração entre estruturas de dados, algoritmos e interface gráfica, produzindo uma ferramenta funcional para fins acadêmicos e demonstrativos.

O sistema transforma conceitos teóricos de grafos em uma aplicação prática de mobilidade urbana.

Limitações da Solução

- Os pesos das arestas são estimativas de tempo, não medições em tempo real.
- O layout visual não representa fielmente a geografia do mapa oficial.
- A centralidade de intermediação é aproximada por caminho único.
- O volume real de passageiros não é considerado na ponderação.
- Eventos externos, atrasos e lotação não são integrados ao sistema.

As limitações indicam oportunidades para evolução em direção a um sistema operacional real.

Trabalhos Futuros

Como continuidade da pesquisa, podem ser desenvolvidas as seguintes melhorias:

- Integrar dados reais de operação, atrasos e lotação.
- Incluir localização geográfica das estações no mapa.
- Implementar pesos dinâmicos conforme horário e demanda.
- Comparar rotas por múltiplos critérios em uma mesma tela.
- Exportar itinerários e relatórios de desempenho.
- Avaliar usabilidade com usuários reais do transporte público.

Referências I

- [1] Vukan R. Vuchic. **Urban Transit Systems and Technology**. John Wiley & Sons, 2007.
- [2] J. A. Bondy e U. S. R. Murty. **Graph Theory with Applications**. Macmillan, 1976.
- [3] Thomas H. Cormen et al. **Introduction to Algorithms**. 4^a ed. MIT Press, 2022.
- [4] Edsger W. Dijkstra. “A note on two problems in connexion with graphs”. Em: **Numerische Mathematik** 1.1 (1959), pp. 269–271.
- [5] Robert W. Floyd. “Algorithm 97: Shortest path”. Em: **Communications of the ACM** 5.6 (1962), p. 345.
- [6] Stephen Warshall. “A theorem on boolean matrices”. Em: **Journal of the ACM** 9.1 (1962), pp. 11–12.

Referências II

- [7] Companhia do Metropolitano de São Paulo. **Mapa da Rede Metroferroviária**. Acesso em: 16 jul. 2026. 2026. URL: <https://www.metro.sp.gov.br/>.
- [8] Companhia Paulista de Trens Metropolitanos. **Mapa da Rede**. Acesso em: 16 jul. 2026. 2026. URL: <https://www.cptm.sp.gov.br/>.
- [9] Omar Elgabry. **Path Finding Algorithms**. Acesso em 16 jul. 2026. 2016. URL: <https://medium.com/omarelgabrys-blog/path-finding-algorithms-f65a8902eb40>.

Obrigado(a) pela Atenção!

Contato:

deborah@ufrb.edu.br

eduardomenezes@aluno.ufrb.edu.br

gabriel salermo@aluno.ufrb.edu.br

gustavo celestino@aluno.ufrb.edu.br

lipesande@aluno.ufrb.edu.br

lincolnms@aluno.ufrb.edu.br